RAJALAKSHMI ENGINEERING COLLEGE

Department of Computer Science and Engineering (Cyber Security)

COURSE NAME: WEB PROGRAMMING

ACADEMIC YEAR: 2026-2027

Project Name: HomeBite (Food Ordering and Operations)

NAME: Mahalaxmi VPK
REG NO: 241901052

PROBLEM STATEMENT / OBJECTIVE

Problem Statement:

- Homemade food sellers may manage menus and orders manually through calls or messages.
- Manual tracking can lead to missed orders, quantity mistakes, and difficulty updating availability.
- Customers need a simple way to browse dishes, choose quantities, and submit delivery or pickup details.

Objectives:

- Display available dishes with descriptions, categories, prices, and images.
- Allow customers to search/filter dishes, manage a cart, and place orders.
- Store orders and menu data in a MySQL database.
- Provide an admin interface to manage menu items and update order status.

TOOLS, TECHNOLOGIES AND IMPLEMENTATION

- React - Builds reusable UI components such as food cards, cart, checkout, and admin views.
- JavaScript-Handles state, filtering, quantity updates, form submission, and API calls.
- Bootstrap + CSS-Responsive layout, styling, icons, and interface components.
- PHP Rest-style API - Receives HTTP requests, validates data, and exposes menu/order operations.
- MySQL + PDO - Persists menu items, orders, and order items; prepared statements handle queries.
- XAMPP + Node.js + Vite - XAMPP runs Apache/MySQL/PHP locally; Node/npm and Vite run the React dev server

Implementation

Code Snippets:

```
const API_URL = import.meta.env.VITE_API_URL || 'http://localhost/homemade-food/backend/api.php';

async function request(action, options = {}, id = null) {
  const url = new URL(API_URL);
  url.searchParams.set('action', action);
  if (id !== null) url.searchParams.set('id', id);

  const response = await fetch(url.toString(), {
    headers: { 'Content-Type': 'application/json', ...(options.headers || {}) },
    ...options
  });

  const data = await response.json().catch(() => ({}));
  if (!response.ok) {
    throw new Error(data.error || 'Request failed. Please ensure the backend server is running.');
  }
  return data;
}

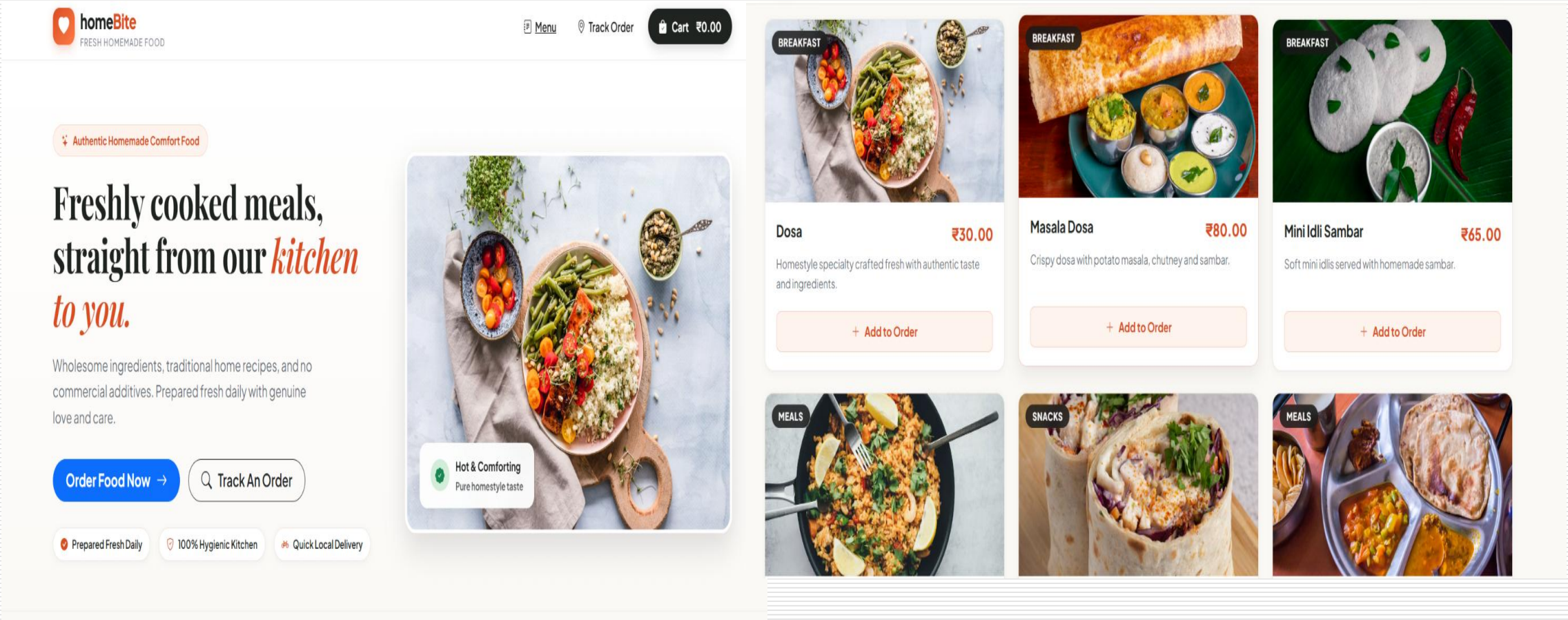
export const api = {
  health: () => request('health'),
  getMenu: () => request('menu'),
  createOrder: (order) => request('orders', { method: 'POST', body: JSON.stringify(order) }),
  trackOrder: (id) => request('order_status', {}, id)
};
```

Implementation

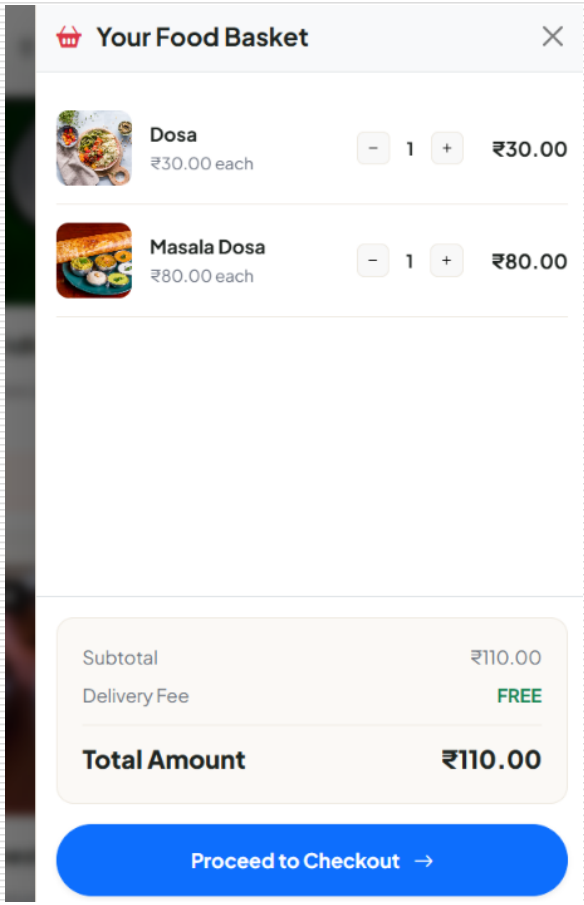
```php
if ($method === 'POST') {
    $stmt = $pdo->prepare('INSERT INTO menu_items (name, description, category, price, image_url, available) VALUES (?, ?, ?, ?, ?, ?)');
    $stmt->execute([$name, $description, $category, $price, $image, $available]);
    respond(['message' => 'Menu item created', 'id' => (int)$pdo->lastInsertId()], 201);
}
$id = (int)($_GET['id'] ?? 0);
if ($id < 1) respond(['error' => 'A valid item id is required.'], 422);
$stmt = $pdo->prepare('UPDATE menu_items SET name=?, description=?, category=?, price=?, image_url=?, available=? WHERE id=?');
$stmt->execute([$name, $description, $category, $price, $image, $available, $id]);
respond(['message' => 'Menu item updated']);
```

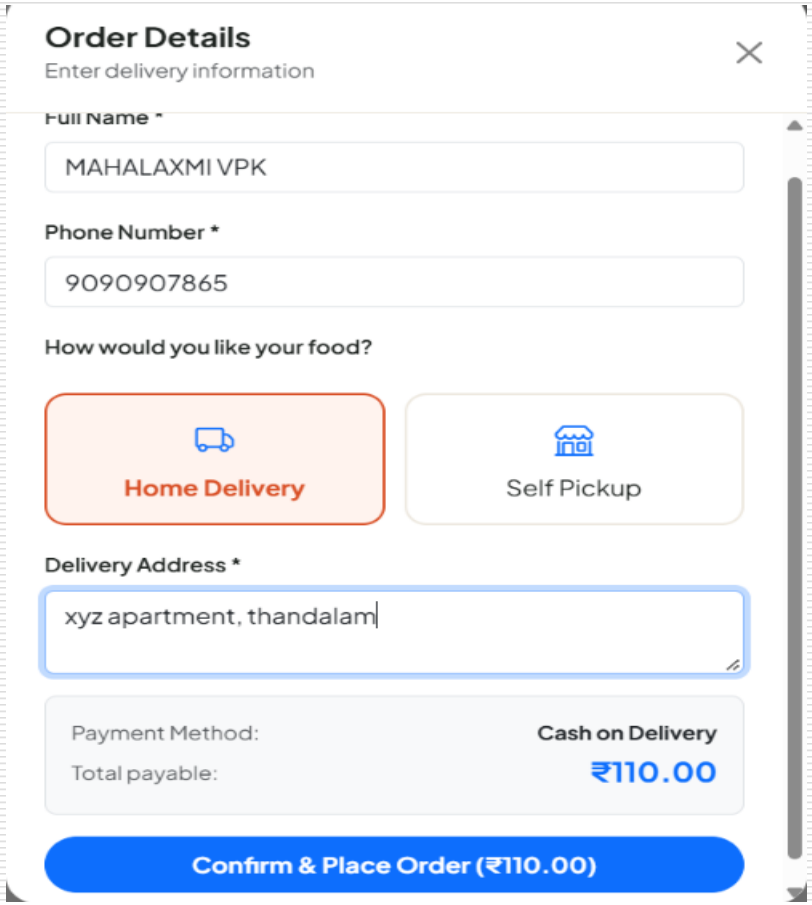
```javascript
// Cart operations
const cartCount = useMemo(() => cart.reduce((sum, x) => sum + x.quantity, 0), [cart]);
const cartTotal = useMemo(() => cart.reduce((sum, x) => sum + Number(x.price) * x.quantity, 0), [cart]);
```

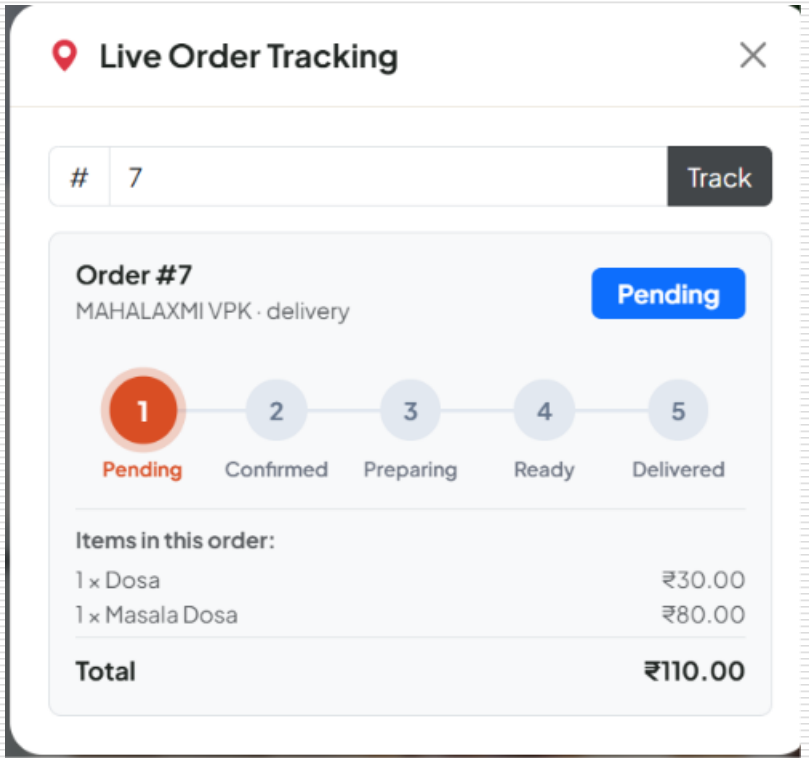
Project Screenshots



Project Screenshots

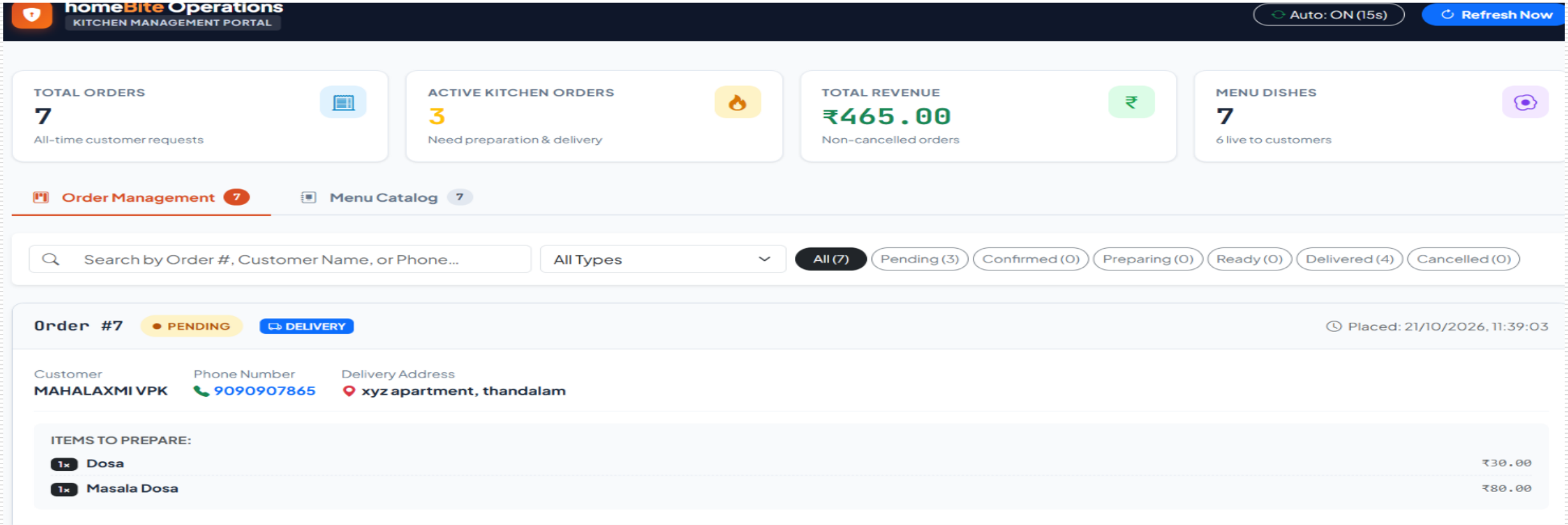




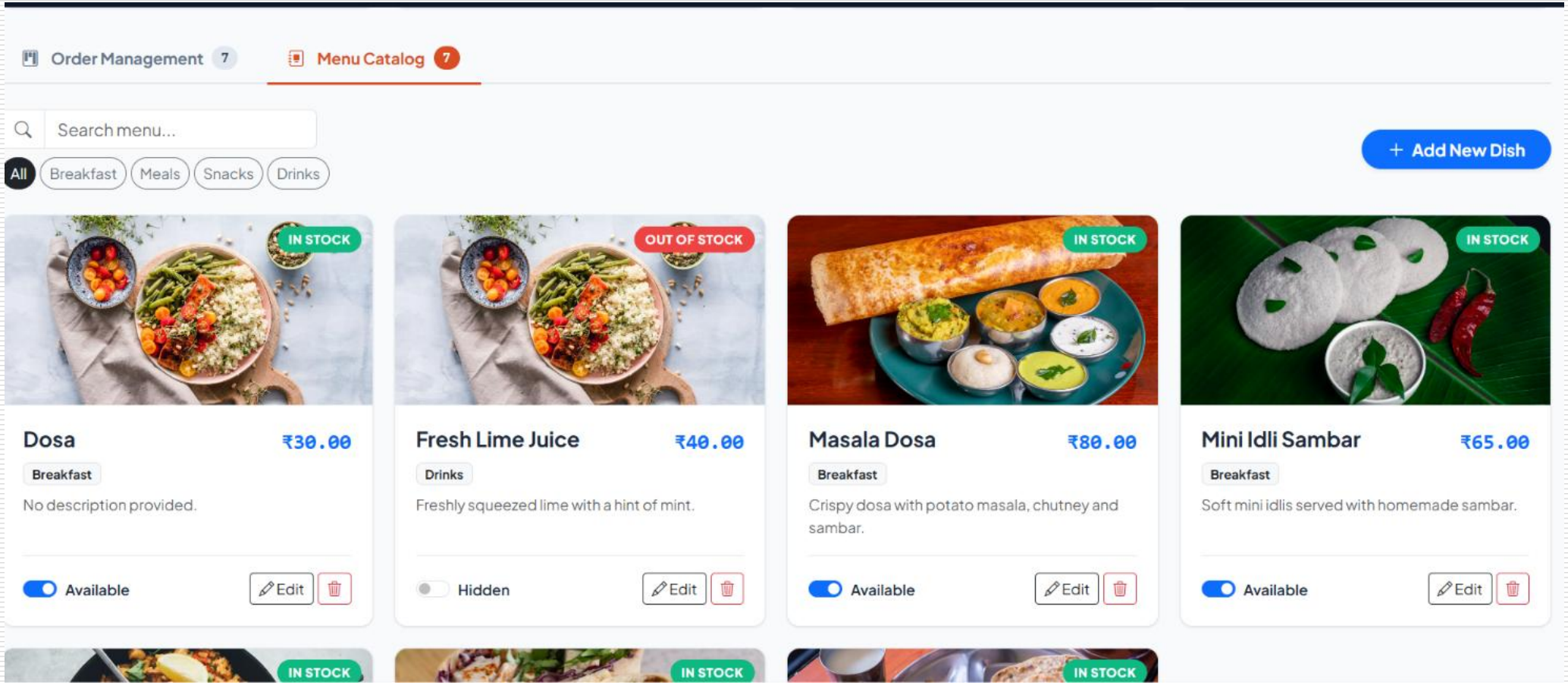


Project Screenshots:

- Admin Dashboard:



Project Screenshots



OUTCOME

- Browse menu items with category filtering and text search.
- Add items to cart, change quantities, remove items, and calculate totals.
- Submit pickup or delivery orders and receive an order number.
- Admin can add, edit, and delete menu items and update order statuses.
- Dashboard statistics summarize products, orders, pending orders, and revenue.